Get 모듈 클라이언트 구현 분석

개요

iroh-blob의 Get 모듈은 클라이언트 측에서 데이터를 요청하고 수신하는 핵심 구성 요소입니다. 이 모듈은 유한 상태 머신(FSM) 패턴을 사용하여 복잡한 데이터 전송 과정을 체계적으로 관리하며, BLAKE3 해시 기반의 실시간 검증과 스트리밍 데이터 수신을 지원합니다.

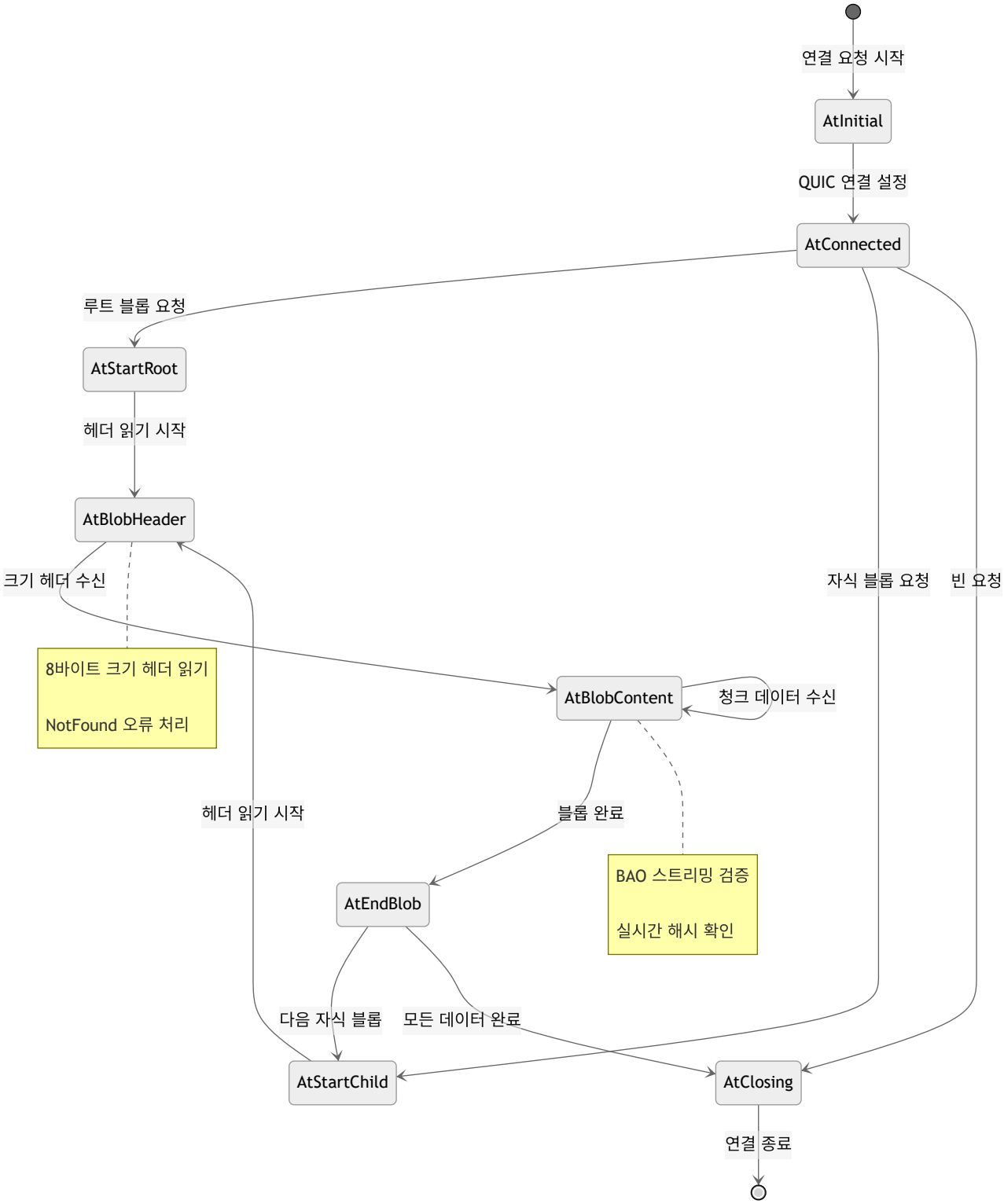
유한 상태 머신(FSM) 아키텍처

FSM 설계 철학

Get 모듈의 FSM은 다음과 같은 설계 원칙을 따릅니다:

1. **명시적 상태 전환**: 각 상태는 명확히 정의된 조건에서만 다음 상태로 전환됩니다
2. **타입 안전성**: Rust의 타입 시스템을 활용하여 잘못된 상태 전환을 컴파일 타임에 방지합니다
3. **오류 격리**: 각 상태에서 발생할 수 있는 오류를 명확히 정의하고 처리합니다
4. **리소스 관리**: 각 상태는 필요한 리소스만 보유하여 메모리 효율성을 보장합니다

상태 다이어그램


연결 요청 시작
AtInitial
QUIC 연결 설정
AtConnected
루트 블롭 요청
AtStartRoot
헤더 읽기 시작
AtBlobHeader
크기 헤더 수신
자식 블롭 요청
빈 요청
8바이트 크기 헤더 읽기
NotFound 오류 처리
AtBlobContent
청크 데이터 수신
헤더 읽기 시작
블롭 완료
BAO 스트리밍 검증
실시간 해시 확인
AtEndBlob
다음 자식 블롭
모든 데이터 완료
AtStartChild
AtClosing
연결 종료

핵심 상태 구조체 분석

AtInitial - 초기 상태

```
pub struct AtInitial {
connection: Connection,
request: GetRequest,
counters: RequestCounters,
}
```

역할과 책임:

- QUIC 연결과 요청 정보를 보관
- 양방향 스트림 생성을 위한 준비 상태
- 성능 카운터 초기화

상태 전환:

```
pub async fn next(self) -> Result<AtConnected, endpoint::ConnectionError>
```

이 메서드는 `connection.open_bi().await` 를 호출하여 새로운 양방향 스트림을 생성하고, 성공 시 `AtConnected` 상태로 전환합니다.

AtConnected - 연결 설정 완료

```
pub struct AtConnected {
start: Instant,
reader: WrappedRecvStream,
writer: SendStream,
request: GetRequest,
counters: RequestCounters,
}
```

핵심 기능:

1. **요청 직렬화 및 전송**: postcard 형식으로 GetRequest를 직렬화
2. **메시지 크기 검증**: MAX_MESSAGE_SIZE(100MiB) 제한 확인
3. **스트림 완료**: writer.finish()로 요청 전송 완료 신호

상태 분기 로직:

```
Ok(match misc.ranges_iter.next() {
Some((offset, ranges)) => {
if offset == 0 {
AtStartRoot { /* ... */ }.into()
} else {
AtStartChild { /* ... */ }.into()
}
}
None => AtClosing::new(misc, reader, true).into(),
```

```
})
```

AtStartRoot - 루트 블롭 시작

```
pub struct AtStartRoot {
ranges: ChunkRanges,
reader: TokioStreamReader<RecvStream>,
misc: Box<Misc>,
hash: Hash,
}
```

특징:

- 컬렉션의 첫 번째 요소 또는 단일 블롭 처리
- 해시가 요청에서 이미 알려져 있음
- `AtBlobHeader` 로 직접 전환 가능

AtStartChild - 자식 블롭 시작

```
pub struct AtStartChild {
ranges: ChunkRanges,
reader: TokioStreamReader<RecvStream>,
misc: Box<Misc>,
offset: u64,
}
```

특징:

- 컬렉션의 자식 블롭 처리
- offset을 통해 현재 처리 중인 자식의 위치 추적
- 외부에서 해시를 제공받아야 `next()` 호출 가능

사용 패턴:

```
let hash = collection.get_child_hash(child.offset());
let blob_header = child.next(hash);
```

데이터 수신 및 검증 과정

AtBlobHeader - 블롭 헤더 처리

```
pub struct AtBlobHeader {
ranges: ChunkRanges,
reader: TokioStreamReader<RecvStream>,
misc: Box<Misc>,
hash: Hash,
}
```

핵심 동작:

1. **크기 헤더 읽기**: 8바이트 little-endian u64 값
2. **NotFound 처리**: EOF 시 데이터 없음으로 해석
3. **ResponseDecoder 초기화**: BAO 트리 구조 설정

```
let size = u64::from_le_bytes(size);
let stream = ResponseDecoder::new(
self.hash.into(),
self.ranges,
BaoTree::new(size, IROH_BLOCK_SIZE),
self.reader,
);
```

AtBlobContent - 스트리밍 데이터 수신

```
pub struct AtBlobContent {
stream: ResponseDecoder<WrappedRecvStream>,
misc: Box<Misc>,
}
```

스트리밍 처리 패턴:

```
pub async fn next(self) -> BlobContentNext {
match self.stream.next().await {
ResponseDecoderNext::More((stream, res)) => {
let mut next = Self { stream, ..self };
let res = res.map_err(DecodeError::from);
match &res {
```

```
Ok(BaoContentItem::Parent(_)) => {
next.misc.other_bytes_read += 64;
}
Ok(BaoContentItem::Leaf(leaf)) => {
next.misc.payload_bytes_read += leaf.data.len() as u64;
}
_ => {}
}
BlobContentNext::More((next, res))
}
ResponseDecoderNext::Done(stream) => BlobContentNext::Done(AtEndBlob {
stream,
misc: self.misc,
}),
}
}
```

BLAKE3 실시간 검증 메커니즘

BAO (BLAKE3 Verified Streaming) 통합

iroh-blob은 bao-tree 크레이트를 사용하여 스트리밍 중 실시간 검증을 수행합니다:

1. **청크 단위 검증**: 1024바이트 청크마다 해시 검증
2. **트리 구조 검증**: 부모 노드 해시를 통한 계층적 검증

3. **부분 데이터 검증**: 범위 요청 시에도 무결성 보장

검증 데이터 구조

```
pub enum BaoContentItem {
Parent(ParentNode), // 64바이트 해시 쌍
Leaf(LeafNode), // 실제 데이터 + 오프셋
}
```

Parent 노드 처리:

- 64바이트 해시 쌍 (left_hash + right_hash)
- 트리 구조 검증을 위한 메타데이터
- `other_bytes_read` 카운터에 반영

Leaf 노드 처리:

- 실제 사용자 데이터
- 오프셋 정보로 정확한 위치 지정
- `payload_bytes_read` 카운터에 반영

검증 실패 처리

```
pub enum DecodeError {
ChunkNotFound {},
ParentNotFound { node: TreeNode },
LeafNotFound { num: ChunkNum },
ParentHashMismatch { node: TreeNode },
```

```
LeafHashMismatch { num: ChunkNum },
Read { source: endpoint::ReadError },
DecodeIo { source: io::Error },
}
```

각 오류 타입은 다음과 같이 분류됩니다:

- **NotFound 계열**: 데이터 부재, 재시도 가능
- **HashMismatch 계열**: 데이터 손상, 제공자 문제
- **Read/IO 계열**: 네트워크 문제, 연결 재시도 필요

오류 처리 및 복구 전략

계층적 오류 분류

Get 모듈은 오류를 다음과 같이 체계적으로 분류합니다:

```
pub enum GetError {
NotFound { source: Box<NotFoundCases> },
RemoteReset { source: Box<RemoteResetCases> },
NoncompliantNode { source: Box<NoncompliantNodeCases> },
Io { source: Box<IoCases> },
BadRequest { source: Box<BadRequestCases> },
LocalFailure { source: Box<LocalFailureCases> },
}
```

오류별 복구 전략

1. NotFound 오류

```
#[snafu(display("Data for hash not found"))]
NotFound {
#[snafu(source(from(NotFoundCases, Box::new)))]
source: Box<NotFoundCases>,
}
```

- **복구 전략**: 다른 제공자에게 재요청
- **발생 상황**: 해시 데이터 부재, 청크 누락
- **처리 방법**: 즉시 재시도 또는 대체 소스 탐색

2. RemoteReset 오류

```
#[snafu(display("Remote has reset the connection"))]
RemoteReset {
#[snafu(source(from(RemoteResetCases, Box::new)))]
source: Box<RemoteResetCases>,
}
```

- **복구 전략**: 연결 재설정 후 재시도
- **발생 상황**: 제공자 재시작, 네트워크 중단
- **처리 방법**: 지수 백오프로 재연결 시도

3. NoncompliantNode 오류

```
#[snafu(display("Remote behaved in a non-compliant way"))]
NoncompliantNode {
#[snafu(source(from(NoncompliantNodeCases, Box::new)))]
source: Box<NoncompliantNodeCases>,
}
```

- **복구 전략**: 해당 노드 블랙리스트 처리
- **발생 상황**: 프로토콜 위반, 잘못된 데이터 전송
- **처리 방법**: 다른 제공자로 즉시 전환

연결 오류 세분화 처리

```
impl From<endpoint::ConnectionError> for GetError {
fn  from(value: endpoint::ConnectionError) -> Self {
use endpoint::ConnectionError;
match  value {
e @ ConnectionError::VersionMismatch => {
NoncompliantNodeSnafu.into_error(e.into())
}
e @ ConnectionError::TransportError(_) => {
NoncompliantNodeSnafu.into_error(e.into())
}
e @ ConnectionError::Reset => {
RemoteResetSnafu.into_error(e.into())
```

```
}
e @ ConnectionError::TimedOut => {
IoSnafu.into_error(e.into())
}
// ... 기타 케이스들
}
}
}
```

이러한 세분화된 오류 처리를 통해 클라이언트는 각 상황에 맞는 최적의 복구 전략을 선택할 수 있습니다.

성능 최적화 기법

메모리 효율적인 스트리밍

청크 단위 처리:

```
match curr.next().await {
BlobContentNext::More((next, item)) => {
match item? {
BaoContentItem::Leaf(leaf) => {
data.write_bytes_at(leaf.offset, leaf.data).await?;
}
BaoContentItem::Parent(parent) => {
```

```
if let Some(outboard) = outboard.as_mut() {
outboard.save(parent.node, &parent.pair).await?;
}
}
}
curr = next;
}
BlobContentNext::Done(end) => return Ok(end),
}
```

장점:

- 대용량 파일도 일정한 메모리 사용량 유지
- 스트리밍 중 실시간 디스크 쓰기 가능
- 네트워크 대역폭과 디스크 I/O 병렬 처리

통계 수집 및 모니터링

```
pub struct Stats {
pub counters: RequestCounters,
pub elapsed: Duration,
}

impl Stats {
pub fn mbits(&self) -> f64 {
```

```
let data_len_bit = self.total_bytes_read() * 8;
data_len_bit as f64 / (1000. * 1000.) / self.elapsed.as_secs_f64()
}
}
```

수집 메트릭:

- `payload_bytes_read` : 실제 사용자 데이터 바이트
- `other_bytes_read` : 검증 메타데이터 바이트
- `elapsed` : 전송 소요 시간
- 전송 속도 (Mbps) 계산

고급 사용 패턴

범위 요청 처리

```
let request = GetRequest::builder()
.root(ChunkRanges::bytes(..1000) | ChunkRanges::bytes(10000..11000))
.build(hash);
```

특징:

- 불연속적인 범위 요청 지원
- 바이트 단위 요청을 청크 단위로 자동 변환
- HTTP Range 요청과 유사한 기능

컬렉션 부분 다운로드

```
let request = GetRequest::builder()
.child(1, ChunkRanges::chunks(1000000..))
.next(ChunkRanges::all())
.build_open(hash);
```

활용 사례:

- 중단된 다운로드 재개
- 선택적 콘텐츠 다운로드
- 대역폭 제한 환경에서의 점진적 로딩

프로브 요청

```
pub async fn get_chunk_probe(
connection: &Connection,
hash: &Hash,
chunk: ChunkNum,
) -> GetResult<Stats>
```

용도:

- 제공자의 데이터 보유 여부 확인
- 네트워크 연결 상태 테스트
- 최소한의 대역폭으로 가용성 검증

실제 사용 예제

기본 블롭 다운로드

```
let connection = endpoint.connect(node_id, iroh_blobs::ALPN).await?;
let mut progress = iroh_blobs::get::request::get_blob(connection, hash);

while let Some(item) = progress.next().await {
match item {
GetBlobItem::Item(BaoContentItem::Leaf(leaf)) => {
// 실제 데이터 처리
process_data(&leaf.data).await?;
}
GetBlobItem::Done(stats) => {
println!("완료: {:.2} Mbps", stats.mbits());
break;
}
GetBlobItem::Error(err) => {
eprintln!("오류: {}", err);
break;
}
}
}
```

저수준 FSM 직접 사용

```
let  request = GetRequest::blob(hash);

let  fsm = fsm::start(connection, request, Default::default());

let  connected = fsm.next().await?;



match  connected.next().await? {

fsm::ConnectedNext::StartRoot(start) => {

let  header = start.next();

let (mut  content, size) = header.next().await?;

loop {

match  content.next().await {

fsm::BlobContentNext::More((next, item)) => {

// 아이템 처리

content = next;

}

fsm::BlobContentNext::Done(end) => {

// 완료 처리

break;

}

}

}

}

_ => { /* 다른 케이스 처리 */ }
```

```
}
```

결론

iroh-blob의 Get 모듈은 유한 상태 머신 패턴을 통해 복잡한 데이터 전송 과정을 체계적으로 관리합니다. BLAKE3 기반의 실시간 검증, 세분화된 오류 처리, 메모리 효율적인 스트리밍 등의 기능을 통해 안전하고 효율적인 데이터 전송을 보장합니다. 이러한 설계는 대용량 파일 전송, 부분 다운로드, 중단된 전송 재개 등 다양한 사용 사례를 효과적으로 지원합니다.